

Bryan Wang
Amrita
CS 314H
September 18, 2026

Program 2 Report

My name is Bryan Wang, and today I will be reporting on my progression through and learning from Project 2 of CS 314H. I will be dividing this report into 4 sections: (1) Overview, (2) Solution Design, (3) Assignment Discussion/Thoughts, (4) Software Testing Methodology, and (5) Social Impact.

Section 1: Overview

The assigned goal of Project 1 was to implement a RandomWriter class that could randomly generate text based on an input file and some integer level, using a Markov-Chain-like process where the last level characters in the output string would be used to generate each next character. Personally, I wanted to gain more experience with writing my own test cases and input validation, which I didn't have to deal with in the prog1.

Section 2: Solution Design

My solution used a HashMap to generate each character. The HashMap contained String keys mapped to Character ArrayList values. The keys in the HashMap were every substring of length level (L) in the input. The ArrayList for each key was every character that came directly after an instance of that substring (key).

To create the HashMap (which I did in the readText function), I did the following:

1. Use a BufferedReader to read each line, and add it to a StringBuilder instance variable called allText. I would add a space to the end of each line to make transitions between lines smoother.
2. Append the first L characters to another StringBuilder instance variable called currentString, which would store the current substring of allText that would be added as a key into the HashMap. I used a StringBuilder for this to make updating currentString easier.
3. Iterate index i from L to the end of allText (-1 because the last character was an extra space I added). During each iteration, I would

- a. Add currentString (as a String instead of a StringBuilder) to an arrayList of keys called keys
- b. If the HashMap did not already contain currentString, I would create a new entry in the Map using currentString (as a String) as the key and an empty Character ArrayList as the value
- c. I would add the current character I'm on to the arraylist corresponding with the current key
- d. I would update currentString by adding the current character to it and deleting the first character. StringBuilder helped with this because it is fast and easy to add/remove characters from
- e. Move on to the next character and repeat the process

This would create a HashMap with every substring in the input, each paired with an arraylist of every character that came immediately after each instance of that substring in the input. When writing outputs, I would do the following:

1. Make currentString a new StringBuilder containing a random starting substring of length L (chosen randomly from the keys arraylist). This is what the output would start with.
2. Create a new StringBuilder called output to store the final output. Add characters from currentString one by one to output, making sure it doesn't get longer than the length variable (output length limit)
3. Iterate index i from L to length, doing the following at each iteration:
 - a. First check if the map didn't contain currentString (saved it as a String called curString). If it didn't, I would reset currentString (and curString) with a new random key to ensure they were a key that existed in the HashMap
 - b. Pick a random Character from the ArrayList associated with curString (currentString) in the map and append it to the output
 - c. Update currentString by appending the character at index i to its end and removing the first character.
 - d. Move on to the next character and repeat the process

This would continuously use the last L characters of the output with the map to generate new characters. These new characters would be based on the input String, ensuring they were in order.

The constructor instantiates all the instance variables (alltext, currentString, map, Level, keys). The main function first calls a helper function called argCheck that makes sure the arguments are valid. It then creates a RandomWriter object, then calls readText and then writeText on it.

This implementation assumes that the input takes a String array with 4 arguments in this order: input file path, output file path, analysis level k, and output file length. It also assumes that the input file can be read line by line and doesn't contain any corrupted sections, and that the output file already exists and can be written to. However, if the output file doesn't exist, it will create one at the specified path. The code checks that 4 arguments were given, that the analysis level and output length are both non negative integers, that the source path is readable, that the output path can be written to, and that the analysis level is less than the length of the input file.

After discussion on Friday, I realized my implementation of readText and writeText are both incorrect. Since each line has a /n newline character, I don't need to add a space between characters in readText, and therefore I don't need to iterate to size-1 in my loop. I also need to add test cases for multi-line inputs. I also realized that my output shouldn't include the key at the beginning of the text when generating characters.

When making these changes, I found an issue: BufferedReader's readLine function automatically strips the \n from each line. I changed it to use the read function instead, which reads character by character but does not ignore \n chars. Read also needs to be cast to characters or it defaults to the character values. BufferedReader's ready function also doesn't account for characters, so I had to change my loop to assign an integer to br.readLine and check if it didn't equal -1 (if it equals -1 that means there are no more characters to read), then cast that int into a character to add it to a StringBuilder. Very tedious process, next time I'll just use Scanner instead.

Section 3: Assignment Discussion/Thoughts

The most difficult part of this project in my opinion was figuring out how to test randomly generated text. I was used to black box testing from prog1 and quickly found that it wouldn't work for this project, and decided to instead check individual parts inside my program to validate outputs.

When writing testReadText(), I decided to check if the map created by readText() matched an expected map for my test input. Since the map was built from the input, I assumed checking the map would validate the input. This helped me catch an error where my map had an extra entry. I used VS Code debugger to help me debug, and realized that the issue came from my input reader adding an extra space at the end of each line, leading to an extra space at the end. I fixed this by stopping the readText loop at alltext.length()-1.

Testing writeText() was more challenging. My original code assumed the current seed would always exist in the map, but sometimes the generated text would randomly create a end

sequence that wasn't in the map. I added a check to pick a new random seed when this happened. However, this implementation means that after resetting, the next character will depend on a new random seed instead of the last characters in the output. These resets make the range of valid outputs much larger.

For example, suppose we have input "abc abd". Technically, output "abddddddddddddd" would be valid because after selecting "bd" (which doesn't have a letter that follows it in the input), the random seed chosen could be "ab" and subsequent letter "d" could be chosen. And since "dd" also isn't in the input, the random seed could choose "d" again and continue the cycle. While this is extremely unlikely, it still means I would have to find a way to validate super rare outputs like this, which I found extremely difficult.

Eventually, I realized that my existing map check already ensured that the output probabilities were correct, since it would also check if each arraylist was correct. I added a short check to see if the output lengths were correct and the characters were all the same as the input, and that was enough. The key was making sure the map was perfectly correct, since a correct map ensured both the outputs were valid and in the correct probabilities.

When writing argument tests, I caught a lot of edge cases that wouldn't work with my code. To make my code work with level 0, I made sure the code appended the next character before deleting the first one, so that even at level 0 `currentString` would just be an empty string. I also moved all my validations from the main function to a separate `checkArgs` helper function to make output validation easier, since the helper function could throw exceptions that the main function would catch and then terminate the program. This allowed me to validate my arguments directly with the `checkArgs` function instead of the main function, which wouldn't work when it terminated. Using `assertThrows` also had me use lambdas, which I had never used before. I learned how lambdas allow functions with errors to not get ran during argument checks, preventing them from throwing an exception before the error-handling function can cover it.

One advantage of my code is that storing the input in a map makes generating the output faster, since it doesn't have to re search through the entire input for every character. Keeping repeated characters in an `ArrayList` can lead to some inefficiencies though since the arraylist can end up storing tons of repeated characters and taking up a lot of memory. One way to fix this would be by creating a custom class that stored the counts of each characters (maybe using another hashmap) as well as the total number of characters, then mapping that to each key. This would prevent repeated characters from getting stored.

My current implementation has some limitations. My text generation includes the first key, so runs with a high level and low output length may end up as just a partial key from the

input. I also don't clear a lot of my instance variables, so calling `readText` more than once on the same object ends up mixing data. `checkArgs` also reads through the entire input to determine its length, which gets repeated when `readText` processes the map. My output validation could also be more rigorous, it currently mostly checks if the map is correct and assumes the generation process using the map is accurate, with some minor checks at the end for good measure.

Section 4: Software Testing Methodology

For this project, I mainly used white box testing, inspecting the hashmap generated by the program instead of its final output. Since the output is random, it's hard to compare it against a specific expected output. It's also difficult to test if the random probabilities for each character are correct using black box testing, unless you run the program many times and compile the results (even then there's still random differences due to variance). I would briefly check the output and see if it had the right number of characters and only used characters in the input.

Most of my test cases tested the argument validation, seeing how the program responded to inaccurate inputs. I would also test for some potential edge cases, such as level 0, output length 0, the output length being less than level, and inputs with lots of duplicates. When debugging, I used the VS Code debugger in the testing menu (the little beaker icon on the left bar) and would go line by line to see where my program went wrong. This approach helped me catch the extra space error as well as the level 0 error (deleting from `currentstring` before adding a new char to it).

I also included some cases of black box testing for small inputs with deterministic (or only a few possible) outputs. For example, I tested a newline case where the file only had 3 newlines. When validating it, I had to make sure the validation type and file type matched up: if it was CRLF, it would contain `/r/n` on each line and total to 6 characters, which would mess up my validation if I expected it to be LF (only `/n` on each line). Learning about this was very annoying.

Section 5: Social Impact

The RandomWriter I implemented for this project is much much simpler than LLMs. However, one interesting thing it demonstrates is that by analyzing larger and larger sections from existing text, you can generate more and more accurate text that eventually

becomes full English words/phrases, albeit not grammatically correct. This helps me better understand how language models came to be, as more advanced language analysis methods than the simple probability approach in this project could be used to create coherent and meaningful text. But that doesn't mean the text is necessarily accurate, AI can (and often has) created inaccurate text that misleads people. It's important to remember that at their core, LLMs are just (very advanced) algorithms that use patterns to generate text. People should be aware that LLMs can and will make mistakes, and that they shouldn't become completely reliant on them (make sure to fact check AI).